

Python, and What It Costs

CONTENTS

01	How Python runs your code	interpreter, typing
02	The basics, in one exercise	functions, loops, assert
03	What a list actually is	references, not values
04	Removing elements	the quadratic trap
05	When a list is the wrong container	sets, hashing
06	Dictionaries	keys and values
07	Mutable, immutable, and keys	tuples
08	The container cheat sheet	costs at a glance

Where the cost hides

Python is a comfortable language with a slow implementation, and the containers it gives you have wildly different performance for operations that look identical in the source. Choosing the wrong one does not produce an error. It produces a program that works on the example and is unusable on real data.

Telling those apart is what the eight sections below are about: how the interpreter runs code and why that makes it slow, what a list actually stores, which operations on one are cheap and which are quadratic, and when a set or a dictionary is the right container instead. Everything is ordinary Python, and none of it needs a library.

01

How Python runs your code

Knowing roughly what happens between pressing run and seeing output explains most of Python's behaviour, including why it is slow and why some errors appear only when a line is actually reached.

FROM SOURCE TO OUTPUT

1. **Lexical and syntactic analysis.** The text is checked against the grammar. A `SyntaxError` here means nothing ran at all, not even the lines above the mistake.
2. **Translation to bytecode**, a compact intermediate form.
3. **Execution.** The interpreter, a virtual machine, reads bytecode instructions one at a time. Each one is implemented by a piece of compiled C code that then runs.

Only during that third step are most semantic rules checked, such as whether an operation supports the types it was handed. That is why a type error inside a branch that never runs will never be reported.

An ahead-of-time compiler, as used by C or Rust, analyses the whole program before running any of it. That costs time up front and buys two things: whole classes of error are caught before the program runs, and the compiler has the opportunity to optimise. Python trades both away for the ability to start immediately. The difference in execution speed can be enormous, which is why serious numerical work in Python is done through libraries whose inner loops are not written in Python at all.

DYNAMICALLY TYPED, AND STRONGLY TYPED

These are two different claims and it is worth keeping them apart.

```
my_variable = 42
my_variable = "abc"           # fine: a name is not tied to a type
```

Dynamic typing means a variable is just a name holding a reference to an object, and it can be pointed at an object of any type at any time. The

type belongs to the object, never to the name.

```
42 + "42"                                # TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Strong typing means Python will not quietly invent a conversion to make that work. Some languages would produce "4242" or 84 ; Python refuses and asks you to say which you meant.

```
str(42) + "42"          # '4242'    string concatenation
42 + int("42")          # 84        numeric addition
```

NOTEBOOKS KEEP THEIR STATE, AND IT WILL CATCH YOU OUT

Running a script starts a fresh interpreter every time and throws it away at the end. A notebook does not: the kernel stays alive between cells, holding every variable and definition from every cell you have run, in the order you ran them rather than the order they appear.

Two consequences. A cell containing something like `counter += 1` gives a different answer each time it is run. And code you have deleted from a cell is still defined in the kernel, so the notebook keeps working until someone opens it fresh, at which point it does not.

Before you submit anything, restart the kernel and run every cell from the top. If it fails then, it was already broken.

02

The basics, in one exercise

Write `is_prime(n)`, returning `True` when `n` is prime and `False` otherwise. Anything at or below 1 is not prime.

```
is_prime(29) -> True
is_prime(1)  -> False
is_prime(4)  -> False
```

Almost every piece of everyday Python appears in this one function: a definition with a parameter and a return value, indentation as block structure, conditionals, a loop, an operator, an import, and a way to test the result.

```
def is_prime(n):
    """Return True if n is a prime number."""
    if n <= 1:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    i = 3                                # check odd divisors only
    while i * i <= n:
        if n % i == 0:
            return False
        i += 2

    return True
```

Three things about that loop are worth stating explicitly.

- **The bound.** Testing divisors up to `n` is wasteful. If `n` has a divisor larger than its square root, it must also have the matching one below it, so stopping at the square root loses nothing. `i * i <= n` expresses that without computing a square root at all.
- **Even numbers are handled once, up front**, which is what allows the loop to step by 2.
- **The edge cases are the exercise.** Zero, one and negative numbers are where a first attempt usually fails, and they fail silently by returning something plausible.

THE SAME THING WITH A FOR LOOP

Every Python `for` loop is a for-each loop. There is no C-style three-part form. To count, you loop over something that produces numbers, which is what `range` is for: `range(start, stop, step)`, with the stop value excluded.

```
import math                                # or: from math import sqrt

def is_prime(n):
    if n <= 1:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    end = int(math.sqrt(n)) + 1             # int() truncates, so + 1 keeps the last candidate
    for i in range(3, end, 2):
        if n % i == 0:
            return False

    return True
```

`math.sqrt` returns a float, and `range` requires integers, so the conversion is not optional. Leaving it out raises `TypeError`. The `+ 1` is there because `int()` truncates towards zero: for a perfect square such as 49, `int(sqrt(49))` is exactly 7, and without the `+ 1` the exclusive upper bound would skip the divisor that matters.

CHECKING YOUR WORK WITH ASSERT

`assert condition, message` does nothing when the condition is true and raises `AssertionError` with your message when it is false. It is the cheapest possible test and it is the right tool for experimental code, where the aim is for mistakes to be loud rather than for the program to survive them.

```
assert is_prime(2) is True, "2 is the smallest prime"
assert is_prime(29) is True, "29 is prime"
assert is_prime(1) is False, "1 is not prime"
assert is_prime(0) is False, "0 is not prime"
assert is_prime(-7) is False, "negative numbers are not prime"
assert is_prime(9) is False, "9 = 3 x 3"
print("all tests passed")
```

INDENTATION IS SYNTAX

Indentation is how Python delimits a block, so it carries the meaning that braces carry elsewhere. Be consistent within a block, and never mix tabs and spaces: the result can look correct and parse as something else entirely. Four spaces is the convention.

03

What a list actually is

A Python list will hold anything, including itself. Understanding why explains both its convenience and its cost.

```
my_list = [42, "Hello World!", None, 3.14159, False]
my_list.append(my_list)

my_list          # [42, 'Hello World!', None, 3.14159, False, [...]]
```

A list does not store values. It stores a contiguous block of **references**, which is to say memory addresses of objects that live elsewhere. That is why the element types need not agree, and why a list can contain itself: it holds its own address, not a copy of itself, so there is no infinite regress. The `[...]` in the output is Python noticing the cycle and declining to recurse.

The block being contiguous is what makes indexing fast. To find item `x[i]`, Python does arithmetic on the starting address rather than walking the list, so `x[i]` costs the same whether the list holds ten items or ten million.

The convenience is paid for in three ways, and they matter once the data gets large:

- Reading one element means two memory accesses, one for the reference and one for the object it points at.
- Every element is a full Python object with its own header, so a list of a million integers is far larger than a million integers.
- Those objects are scattered in memory rather than packed together, which defeats the processor cache.

This is precisely the cost an array library such as NumPy removes, by storing raw values of a single type packed end to end instead of references to objects scattered across memory.

04

Removing elements, and the quadratic trap

Given a list of integers, remove every negative value.

```
[4, -1, 7, -9, 0, -3, 5] -> [4, 7, 0, 5]
```

The obvious approach walks the list and deletes what it does not want. It is correct, and it is the slow one.

```
def remove_negatives(numbers):
    nums = numbers.copy()          # see the note below on why
    i = 0
    while i < len(nums):
        if nums[i] < 0:
            nums.pop(i)            # do not advance i: the next element slid into this slot
        else:
            i += 1
    return nums
```

A `while` loop is needed rather than a `for` loop, because the length changes as you go and iterating a list while modifying it is unreliable. That awkwardness is the first hint that the approach is wrong.

The real problem is `pop`. Elements sit in a contiguous block, so removing one from the middle leaves a hole that would break the address arithmetic. Python closes it by shifting every later element down one place, which is $O(n)$ work for a single removal. Doing that once per element makes the whole function $O(n^2)$.

Building a new list instead touches each element once:

```
def remove_negatives(numbers):
    kept = []
    for num in numbers:
        if num >= 0:
            kept.append(num)
    return kept
```

`append` adds at the end, where there is no shifting to do. A list keeps spare capacity beyond its current length, so most appends drop straight into a free slot. When it fills, Python allocates a larger block and copies everything across, but since each new block is proportionally bigger those copies grow rarer as the list grows. Averaged over many appends the cost per append is constant, which is what **amortised $O(1)$** means. The whole function is $O(n)$.

Same answers, and nothing in the source of either says which is which. Timed on one machine, removing the negatives from a list of `n` numbers:

N	POP IN A LOOP	BUILD A NEW LIST	RATIO
10 000	0.002 s	0.0002 s	11x
60 000	0.047 s	0.0006 s	76x
200 000	0.505 s	0.0024 s	213x
500 000	3.210 s	0.0063 s	513x

The absolute seconds depend on the machine and on the Python version, so do not memorise them. The last column is the part that matters: the gap is not a fixed penalty, it **widens as the input grows**. That widening is exactly what $O(n^2)$ against $O(n)$ means, and it is why a version that looks fine on a test case of seven elements can be unusable on real data.

Written as a comprehension, the fast version is one line:

```
kept = [num for num in numbers if num >= 0]
```

WHY THE FIRST VERSION COPIED THE LIST

Arguments are passed by reference. The name `numbers` inside the function refers to the very same list object as the name at the call site, so calling `pop` on it changes the caller's list. A function that quietly modifies what it was given is a hard bug to find later, which is why the slow version starts with `numbers.copy()`.

The fast version needs no copy, because it never modifies the input at all. That is the better fix: prefer building a new value over changing one you were handed.

```
original = [1, -2, 3]
result = remove_negatives(original)
original                                # [1, -2, 3], untouched
```

05

When a list is the wrong container

Given a log of student IDs, find every ID that appears more than once, each reported once.

```
["U102", "U105", "U102", "U108", "U105", "U101"]
-> {"U102", "U105"}
```

The direct approach keeps a list of what it has already seen:

```
def find_duplicates(submissions):
    seen = []
    duplicates = []
    for student_id in submissions:
        if student_id in seen:           # a linear scan
            if student_id not in duplicates: # another linear scan
                duplicates.append(student_id)
        else:
            seen.append(student_id)
    return duplicates
```

Every answer is right. The cost is hidden in `in`. Testing membership of a list compares against elements one at a time, so it is $O(n)$ on average, and `seen` grows as the loop proceeds. Doing an $O(n)$ lookup once per element makes the function $O(n^2)$.

There are two general ways to make lookup faster. Sorting first costs $O(n \log n)$ and lets you binary search afterwards. Hashing computes a number from the value itself and uses it to jump straight to where the value would be, which is $O(1)$ on average. Python's `set` and `dict` both do the second.

```
def find_duplicates(submissions):
    seen = set()           # note: {} would give you a dict, not a set
    duplicates = set()
    for student_id in submissions:
        if student_id in seen: # constant time on average
            duplicates.add(student_id)
        else:
```

```
        seen.add(student_id)
    return duplicates
```

One character of difference in the source, and the function is now $O(n)$. The habit worth forming from this: **if you are about to write in against a list inside a loop, the list should almost certainly be a set.**

WHAT A SET IS

A set behaves like a mathematical one:

- It holds no duplicates. Adding something already present does nothing, silently.
- It is unordered. Iteration order is not the insertion order and can change after adding or removing anything, so never rely on it.

Two consequences worth remembering. `set(my_list)` is the shortest way to get the distinct values from a list, and `len(set(x)) != len(x)` answers "are there duplicates" in one line.

EMPTY CONTAINERS

`{}` is an empty dictionary, not an empty set. The literal syntax collides, and the dictionary won. An empty set has to be written `set()`. Non-empty set literals are unambiguous: `{"a", 42}` is a set.

06

Dictionaries

A set answers "have I seen this". A dictionary answers "what is stored under this", which is the question you actually have most of the time.

A dictionary holds key and value pairs. Keys are unique, and looking one up uses the same hash mechanism a set uses, so it is $O(1)$ on average.

```
empty = {}                                # or dict()

stock = {"Produce": ["Apples", "Bananas"],
        "Dairy":   ["Milk", "Cheese"],
        "Bakery":  []}
```

THREE WAYS TO ITERATE

Because each entry is a pair, there are three sensible things to loop over. Looping over the dictionary itself gives the keys, which is the one people forget.

```
for key in stock:                        # keys, same as stock.keys()
    print(key)

for value in stock.values():             # values only
    print(value)

for key, value in stock.items():         # both at once
    print(key, value)
```

ADDING, UPDATING, DELETING

Assignment by key does double duty: it inserts when the key is new and overwrites when it is not. That is convenient and it is also how a typo silently creates a second entry instead of updating the one you meant.

```
stock["Frozen"] = ["Peas"]              # inserts, the key was new
stock["Frozen"] = ["Chips"]             # overwrites, the key existed
del stock["Frozen"]                     # removes the pair
"Frozen" in stock                       # False. membership tests the KEY, not the value
```

Since Python 3.7 a dictionary preserves the order keys were first inserted, and iteration follows that order. A set gives you no such promise. Values may be of any type, keys may not, which is the subject of the next section.

07

Mutable, immutable, and why keys care

Suppose you want to record how long a drone took to fly a route, keyed by the route itself: a sequence of waypoints. The obvious key is a list of coordinates, and it does not work.

```
costs = {}
route = [[15.5, 27.2], [19.5, 21.4], [22.3, 18.1]]
costs[route] = 29                                # TypeError: unhashable type: 'list'
```

THE TWO WORDS

An object is **mutable** if its state can change after it is created, and **immutable** if it cannot.

```
values = [1, 2]
values.append(3)                                # fine: a list is mutable, so it can be changed in place
```

```
text = "abcd"
text[1] = "X"                                   # TypeError: 'str' object does not support item assignment
```

Strings are immutable. Anything that looks like changing one, such as concatenation or replacing a character, actually builds a new string and leaves the old one alone.

WHY A HASH TABLE CANNOT ACCEPT A MUTABLE KEY

Looking up a key means hashing it to decide where to look. Suppose a list were allowed as a key. It hashes to slot 41 and is stored there. Something elsewhere in the program appends to that same list, since you still hold a reference to it. Its hash is now different, say slot 7. The entry has become unreachable: a lookup goes to slot 7 and finds nothing, while the entry sits in 41 forever.

Rather than trust every programmer to avoid that, Python makes it impossible. The mutable built-in types have no usable hash, so they cannot be dictionary keys or set members at all.

TYPE	MUTABLE	HASHABLE	KEY OR SET MEMBER
int , float , str , bool	no	yes	yes
tuple	no	if its contents are	yes
frozenset	no	yes	yes
list	yes	no	no
dict , set	yes	no	no

TUPLES

A tuple is Python's immutable sequence, written with parentheses. It is what the drone example needed:

```
costs = {}
route = ((15.5, 27.2), (19.5, 21.4), (22.3, 18.1))
costs[route] = 29                                # works

single = (42,)                                   # the comma makes it a tuple; (42) is just 42
```

```
route[0] = (0, 0)                                # TypeError: 'tuple' object does not support item assignment
```

Note the qualifier in the table. A tuple is hashable only if everything inside it is, because its hash is computed from its contents. (1, 2) is a fine

key; (1, [2]) raises the same `TypeError`, because the list inside it can still change. Immutability has to go all the way down.

YOU ALREADY USE TUPLES CONSTANTLY

Python packs and unpacks them for you, which is why they are easy to miss.

```
a, b = 1, 2           # both sides are tuples
a, b = b, a           # a swap with no temporary variable

for key, value in stock.items(): # items() yields 2-tuples, unpacked into two names
    ...

def minmax(xs):
    return min(xs), max(xs)      # one tuple is returned, not two values

low, high = minmax([3, 1, 4])
```

Unpacking requires the counts to match on both sides; if they do not, Python raises `ValueError` rather than guessing.

08

The container cheat sheet

The one table worth memorising. Every row is a decision someone makes without noticing.

OPERATION	LIST	TUPLE	SET	DICT
Read by index, <code>x[3]</code>	<code>O(1)</code>	<code>O(1)</code>	no	no
Read by key, <code>x[k]</code>	no	no	no	<code>O(1)</code> avg
Membership, <code>v in x</code>	<code>O(n)</code>	<code>O(n)</code>	<code>O(1)</code> avg	<code>O(1)</code> avg
Add at the end	<code>O(1)</code> amortised	no	<code>O(1)</code> avg	<code>O(1)</code> avg
Insert or delete at the front	<code>O(n)</code>	no	not ordered	<code>O(1)</code> avg
Keeps insertion order	yes	yes	no	yes
Allows duplicates	yes	yes	no	keys no, values yes
Can be a key or set member	no	if contents are	no	no

Read the membership row twice. It is the difference between the two versions of section 5, and it is the one that turns a working program into an unusable one without changing a single answer.

AVERAGE, AND AMORTISED

Average: if many keys land in the same slot they collide, and lookup degrades towards scanning that slot. With ordinary data and Python's own hash functions this does not happen in practice, which is why the average is the figure quoted.

Amortised: most appends are free, an occasional one triggers a copy of the whole list, and because the new block is proportionally larger those copies get rarer as the list grows. Averaged over many appends, the cost per append is constant.

Where to look things up

The standard documentation is the reference worth getting used to, and it reads as well offline as on.

- The tutorial (<https://docs.python.org/3/tutorial/>), for reading through a topic.
- The library reference (<https://docs.python.org/3/library/>) and the language reference (<https://docs.python.org/3/reference/>), for the exact rules.
- The built-in functions (<https://docs.python.org/3/library/functions.html>), a page worth skimming once in full: `len`, `range`, `sum`, `min`, `max`, `sorted`, `enumerate`, `zip`, `int`, `str`, `set`, `dict`.

The short version

A name holds a reference, not a value. A list is a block of references, so indexing is instant and searching is not. Removing from the middle shifts everything after it, so build a new list instead. Membership in a list is a scan and membership in a set is a jump, which is the difference between a program that finishes and one that does not. And anything used as a key has to be immutable all the way down, which is what tuples are for.

